

# Лабораторная работа №1

Шифрам простой замены

---

11 Марта 2026 - Ли Хан

2026

Российский университет дружбы народов, Москва, Россия

## Цель работы

---

я подробно расскажу вам о содержании первой главы лабораторной работы, посвященной «Шифрам простой замены», и на примере написанного мною кода продемонстрирую, как пошагово выполнить практические задания.

В этой главе мы изучили основные принципы функционирования шифров простой замены. Основная идея заключается в том, что символы исходного алфавита заменяются символами шифроалфавита по определенному правилу.

Для выполнения заданий я использовал язык Python. Вот логика моей реализации:

Прежде всего, я определил строку стандартного русского алфавита, состоящую из 33 букв. Это база для всех дальнейших вычислений.

## Шаг 2: Реализация шифра Цезаря (с произвольным ключом $k$ )

В функции `caesar_cipher` я обрабатываю каждый символ по следующему алгоритму:

- Определение индекса: Сначала с помощью функции `find` я нахожу числовую позицию символа в алфавите.
- Выполнение сдвига: К этой позиции прибавляется ключ  $k$ . Чтобы обработать выход за пределы алфавита, я использую операцию взятия остатка:  $(\text{index} + k) \% 33$ . Так, после буквы «я» сдвиг безопасно возвращается к букве «а».
- Восстановление символа: Полученный новый индекс преобразуется обратно в букву с сохранением исходного регистра (заглавная или строчная).

## шифра Цезаря компилятора

[5]  
✓ 0  
秒

```
def caesar_cipher(text, key, alphabet):  
    """  
    此函数实现了凯撒密码的加密与解密逻辑，它是通过将字符在字母表中按固定位移量移动来实现的。  
    Эта функция реализует логику шифра Цезаря, который р  
    """  
  
    result = ""  
    # 首先，我们要获取字母表的总长度，它是我们后续进行循环移位（模运算）的基准。  
    # Во-первых, мы получаем общую длину алфавита, которая  
    m = len(alphabet)  
  
    for char in text:  
        # 我们需要判断当前字符是否在预设的字母表中，从而决定是否对其进行加密处理。  
        # Нам нужно проверить, входит ли текущий символ :  
        if char.lower() in alphabet:  
  
            # 第一步，我们定位该字符在字母表中的原始索引位置，这是计算位移的起点。  
            # Шаг 1: мы находим исходный индекс этого сим  
            idx = alphabet.find(char.lower())  
  
            # 第二步，我们将原始索引与密钥相加，并通过对字母表长度取模，确保位移超出末尾时能回到开  
            # Шаг 2: мы складываем индекс с ключом и испо  
            new_idx = (idx + key) % m  
  
            # 第三步，我们根据计算出的新索引提取加密字符，并判断原字符的大小写状态以保持格式一致。  
            # Шаг 3: мы извлекаем зашифрованный символ по  
            new_char = alphabet[new_idx]  
            result += new_char.upper() if char.isupper() else new_char  
        else:  
            # 如果字符是空格或标点符号，我们则不进行任何修改，直接将其原样拼接到结果中。  
            # Если символ является пробелом или знаком п  
            result += char  
  
    return result
```

```
# --- 演示与测试逻辑 / Демонстрация и тестирование ---

# 我们定义一个完整的俄语字母表，包含33个字母，作为程序运行的基础词库。
# Мы определяем полный русский алфавит из 33 букв,
ru_alphabet = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"

# 这里演示凯撒密码：输入明文和密钥，函数将按照我们解释的位移逻辑生成密文。
# Здесь демонстрация шифра Цезаря: вводим текст и
print("Caesar:", caesar_cipher("Пришел увидел победил", 3, ru_alphabet))

# 这里演示阿特巴什密码：只需输入明文，函数将通过镜像映射生成加密后的结果。
# Здесь демонстрация шифра Атбаш: достаточно ввес
print("Atbash:", atbash_cipher("Саратов", ru_alphabet))

... Caesar: Тулызо целжзо тсдзжло
Atbash: Няоямрэ
```

Рис. 2: шифра Цезаря результат



В функции `atbash_cipher` логика еще более наглядна:

- Создание зеркала: С помощью среза `[::-1]` я быстро генерирую полностью перевернутый алфавит.
- Прямое отображение: Программа находит позицию символа в обычном алфавите и берет соответствующий символ из зеркального алфавита по тому же индексу.

```
def atbash_cipher(text, alphabet):
```

```
    """
```

此函数实现了阿特巴什密码，其核心在于利用一个完全反转的字母表来进行镜像替换。

Эта функция реализует шифр Атбаш, суть ко

```
    """
```

```
    result = ""
```

# 我们首先通过 Python 的切片操作快速创建一个与原始顺序完全相反的镜像字母表。

# Сначала мы создаем зеркальный алфавит с

```
    reversed_alphabet = alphabet[::-1]
```

```
    for char in text:
```

```
        if char.lower() in alphabet:
```

# 第一步，我们找到原字符在标准顺序字母表中的位置索引。

# Шаг 1: мы находим индекс исходно

```
        idx = alphabet.find(char.lower())
```

# 第二步，我们直接从镜像字母表中取出相同位置的字符，从而完成“首

# Шаг 2: мы берем символ из зеркал

```
        new_char = reversed_alphabet[idx]
```

```
        result += new_char.upper() if char.isupper() else new_char
```

```
    else:
```

# 对于不在字母表内的符号（如数字或空格），程序会跳过加密逻辑并保

# Для символов вне алфавита (цифры

```
        result += char
```

```
    return result
```

```
# --- 演示与测试逻辑 / Демонстрация и тестирование ---

# 我们定义一个完整的俄语字母表，包含33个字母，作为程序运行的基础词库。
# Мы определяем полный русский алфавит из 33 букв,
ru_alphabet = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"

# 这里演示凯撒密码：输入明文和密钥，函数将按照我们解释的位移逻辑生成密文。
# Здесь демонстрация шифра Цезаря: вводим текст и
print("Caesar:", caesar_cipher("Пришел увидел победил", 3, ru_alphabet))

# 这里演示阿特巴什密码：只需输入明文，函数将通过镜像映射生成加密后的结果。
# Здесь демонстрация шифра Атбаш: достаточно ввес
print("Atbash:", atbash_cipher("Саратов", ru_alphabet))

... Caesar: Тулызо целжзо тсцзжло
Atbash: Няоямрз
```

Рис. 4: шифра Атбаш результат

| Характеристика     | Шифр Цезаря                | Шифр Атбаш                                            |
|--------------------|----------------------------|-------------------------------------------------------|
| Логика шифрования  | Линейный сдвиг ( $i + k$ ) | Зеркальное отражение                                  |
| Требование к ключу | Нужен параметр сдвига $k$  | Фиксированная логика, ключ не нужен                   |
| Цикличность        | Реализуется через модуль   | Симметричен: повторное шифрование дает исходный текст |

Рис. 5: Таблица сравнения логики

## Итоги работы

---

Следуя этим шагам, я успешно реализовал алгоритмы, требуемые в лабораторной работе. Этот подход обеспечивает точность шифрования и гибкость при работе с любыми текстами и ключами.